**Software Requirement Specification (SRS)**

For

# Project: AI Assisted Smart Library System

**Prepared by: Egiti. Guru Venkata Krishna**

**Registration ID: 24BCA7976**

**School of Computer Science and Engineering**

**Date Created:** January 22, 2026

Guided By: P. Monica

Table of contents

Page Numbers


1. Introduction
   - 1.1 Purpose
   - 1.2 Intended Customers/Users
   - 1.3 Product Scope
   - 1.4 Outline of the document
2. Overall Description
   - 2.1 Product Perspective
   - 2.2 Product Functions
   - 2.3 User Classes
   - 2.4 Design and Implementation Constraints
3. Diagrams
   - 3.1 UML Diagrams
   - 3.2 ER Diagrams
   - 3.3 DFD Diagrams
   - 3.4 Assumptions and Dependencies
4. External Interface Requirements/Screenshots
   - 4.1 User Interfaces
   - 4.2 Hardware Interfaces
   - 4.3 Software Interfaces
   - 4.4 Communication Interfaces
5. Other Non-Functional Requirements
   - 5.1 Performance Requirements
   - 5.2 Safety Requirements
   - 5.3 Security Requirements
   - 5.4 Software Quality Attributes
   - 5.5 Business Policies/Rules (if any)
6. Testing
   - 6.1 Block-Box Testing
   - 6.2 White-Box Testing
7. Conclusion

# 1.Introduction

## 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to document the functional and non- functional requirements for the "NextGen Smart Library System." This document is intended for developers, project stakeholders (guides/evaluators), and future maintainers of the system. It outlines the architectural decisions, specifically focusing on the integration of Artificial Intelligence for resource discovery and temporal logic for resource scheduling.

## 1.2 Intended Customers/Users

The system is designed for the following user classes within an academic institution:

- **Students:** The primary beneficiaries who will use the system to discover study materials and book physical resources.
- **Faculty or Research Scholars:** Users requiring access to niche academic resources.
- **Librarians as Administrators:** Personnel responsible for managing inventory, approving requests, and maintaining system integrity.

## 1.3 Product Scope

The "AI Assisted Smart Library System" is a full-stack web application designed to modernize traditional library operations. Unlike standard CRUD (Create, Read, Update, Delete) systems, this product integrates:

- **AI-Driven Discoverability:** A search engine capable of understanding user intent despite spelling errors (Fuzzy Logic) and recommending books based on content similarity (TF-IDF).
- **Resource Planning:** A rule-based scheduling engine for discussion rooms that prevents conflicts and enforces institutional policies (e.g., ID validation).
- **Administrative Resilience:** A disaster recovery module ("Black Box") to prevent accidental data loss.

## 1.4 Outline of the Document

This document is organized into specific sections: Section 2 describes the general factors that affect the product and its requirements. Section 3 (omitted as per instruction) covers system models. Section 4 details interface requirements. Section 5 specifies non-functional requirements like security and performance. Section 6 outlines the testing strategies used to validate the software.

# 2.Overall Description

## 2.1 Product Perspective

This system is a standalone web-based solution. It is built using the Flask micro-framework (Python) and acts as a replacement for manual ledger systems or legacy library software that lacks "intelligence." It utilizes a flat-file database architecture (CSV) for portability during the prototype phase but is architected to support migration to SQL databases.

## 2.2 Product Functions

The major functions of the system include:

1. Smart Search: Retrieving book results using Natural Language Processing (NLP) to handle typos and context.
2. Recommendation Engine: Suggesting relevant reading materials based on the category and content of the currently viewed book.
3. Temporal Room Booking: Allowing student groups to reserve discussion rooms with strict validation (Time, ID, Concurrency).
4. Inventory Management: Adding, updating, or removing books from the catalog.
5. Disaster Recovery: Automated backup of critical data before bulk administrative actions.

## 2.3 User Classes and Characteristics

**Student User:**

- Technical Skill: Basic web navigation.
- Access Rights: Search, View Recommendations, Book Rooms.
- Constraint: Must possess a valid Registration Number in the format 24bca7976.

**Administrator:**

- Technical Skill: Moderate. Capable of understanding system alerts.
- Access Rights: Full CRUD access to Inventory and Bookings. Capability to override or cancel active bookings.

## 2.4 Design and Implementation Constraints

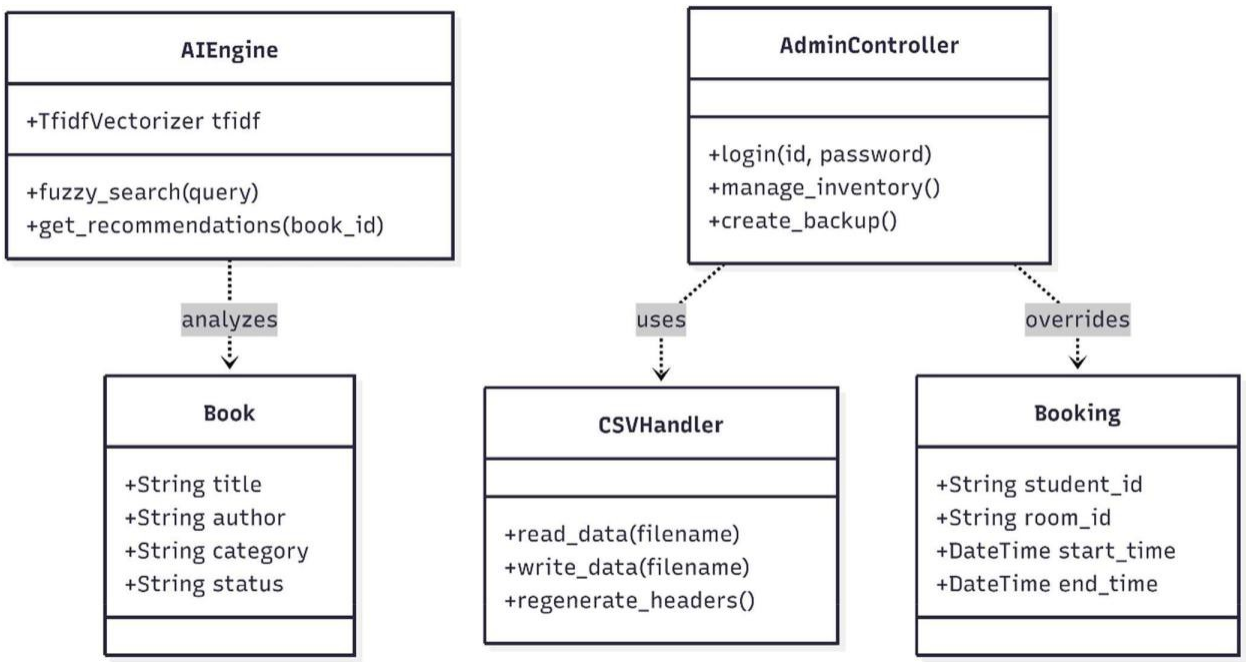
- Hardware: The system must run on standard university lab computers with minimal RAM overhead (AI models are pre-computed).
- Network: Requires an Intranet or Internet connection for the Web UI.
- Persistence: For this version (V1.0), data is stored in CSV format. Concurrent write operations are limited by file-locking mechanisms.
- Time Constraints: Room bookings are strictly limited to the operational hours of 09:00 AM to 18:00 PM.

## 2.5 Assumptions and Dependencies

- It is assumed the hosting server has Python 3.8+ installed.
- The system relies on open-source libraries: scikit-learn, pandas, fuzzywuzzy. It is assumed these libraries remain compatible with the deployment environment.
- User ID format policies (Batch+Branch+Serial) are assumed to remain constant.

## 3.Diagrams (UML, ER, DFD)

### 1.CLASS_DIAGRAM:



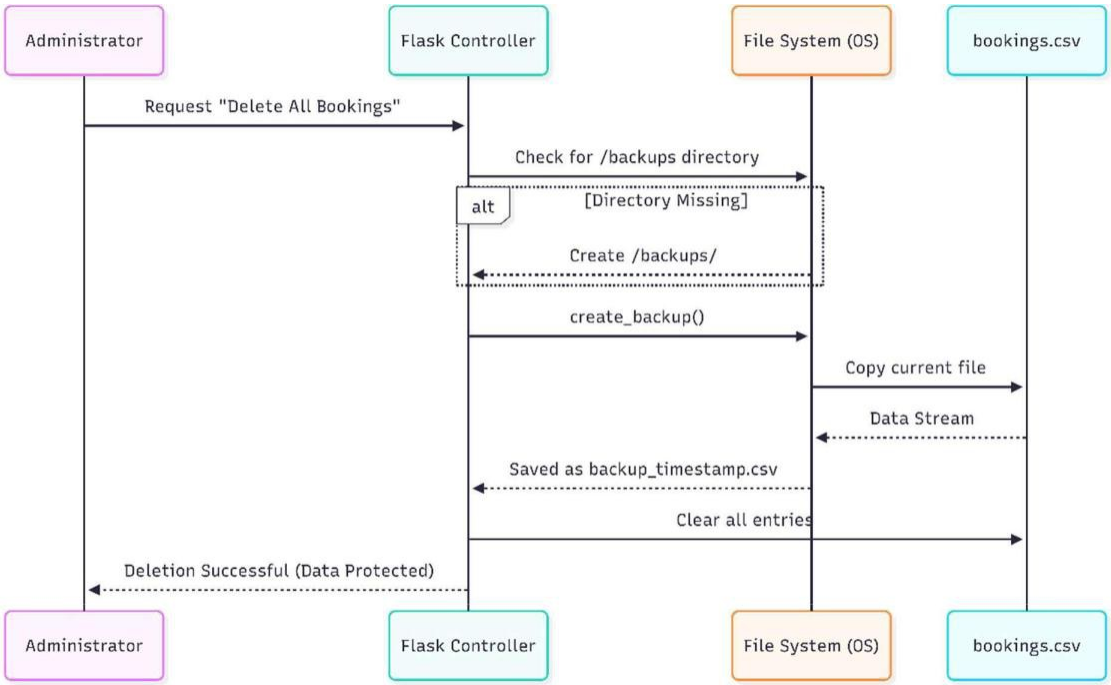

**Description:**

This is a static model that illustrates the internal structure of the software, including its attributes and methods.

**Purpose:**

It shows how the **AI Engine** interacts with the **Book** data and how the **Admin Controller** utilizes the **CSV Handler** to manage data persistence and backups.

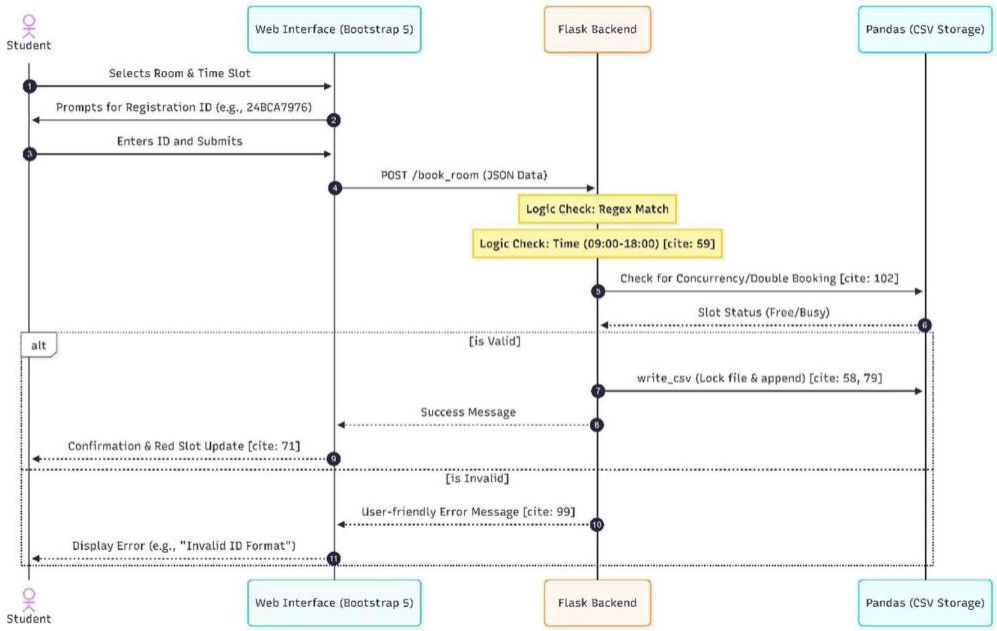
## 2.SEQUENCE_DIAGRAM FOR _RECOVERY:




## <u>Disaster Recovery:</u>

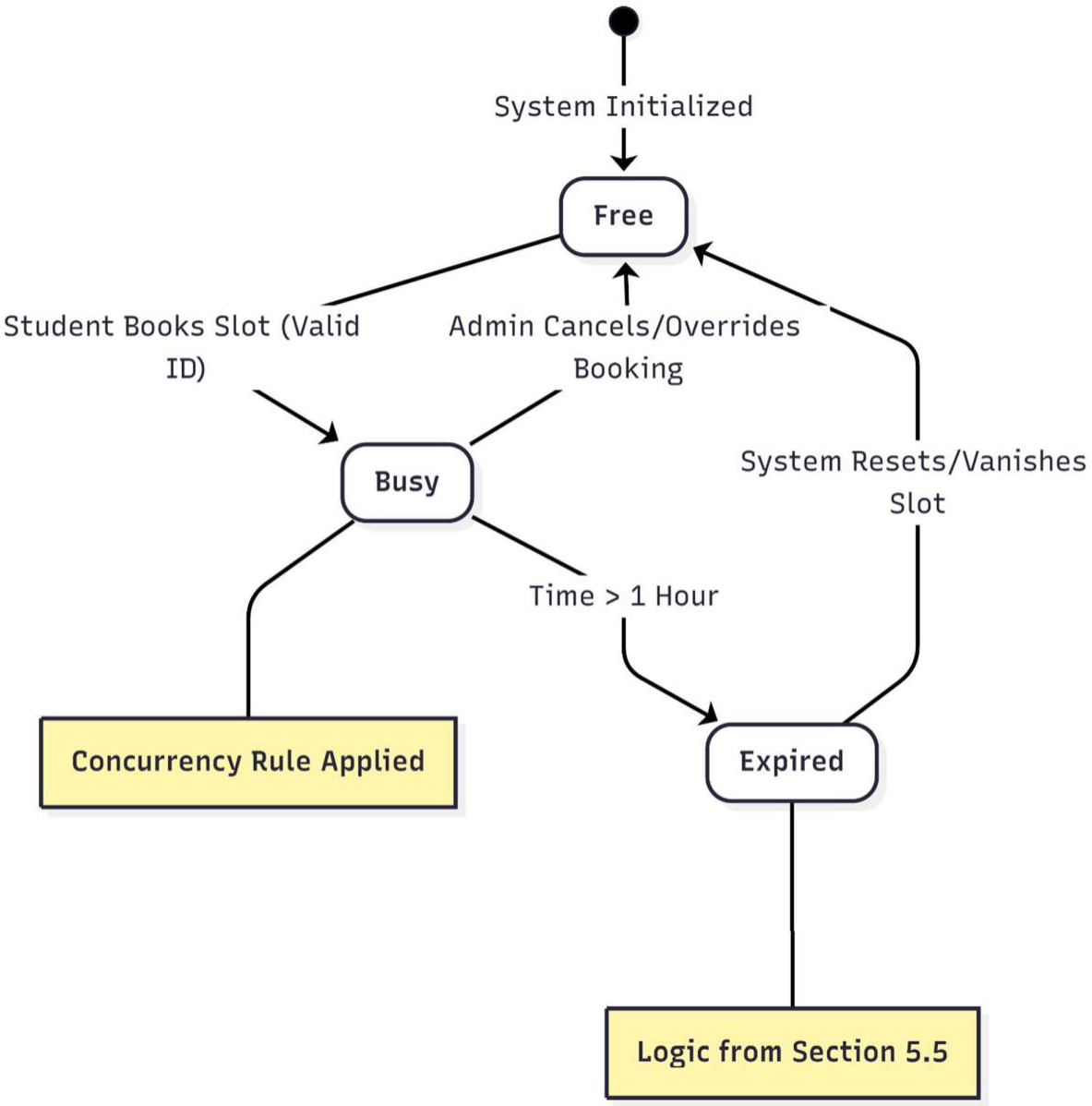
This focuses on the **"Black Box"** safety mechanism, showing how the system creates an automated data backup before executing bulk administrative deletion

## 3 SEQUENCE_DIAGRAM for Discussion Room:




**<u>Discussion Room Booking:</u>** This diagram illustrates the chronological flow of data between the User Interface, the Flask Backend, and the CSV database. It ensures that the **1-hour booking rule** and **operational hour constraints (09:00 - 18:00)** are enforced.

## 4.STATE_MACHINE Diagram (Booking):



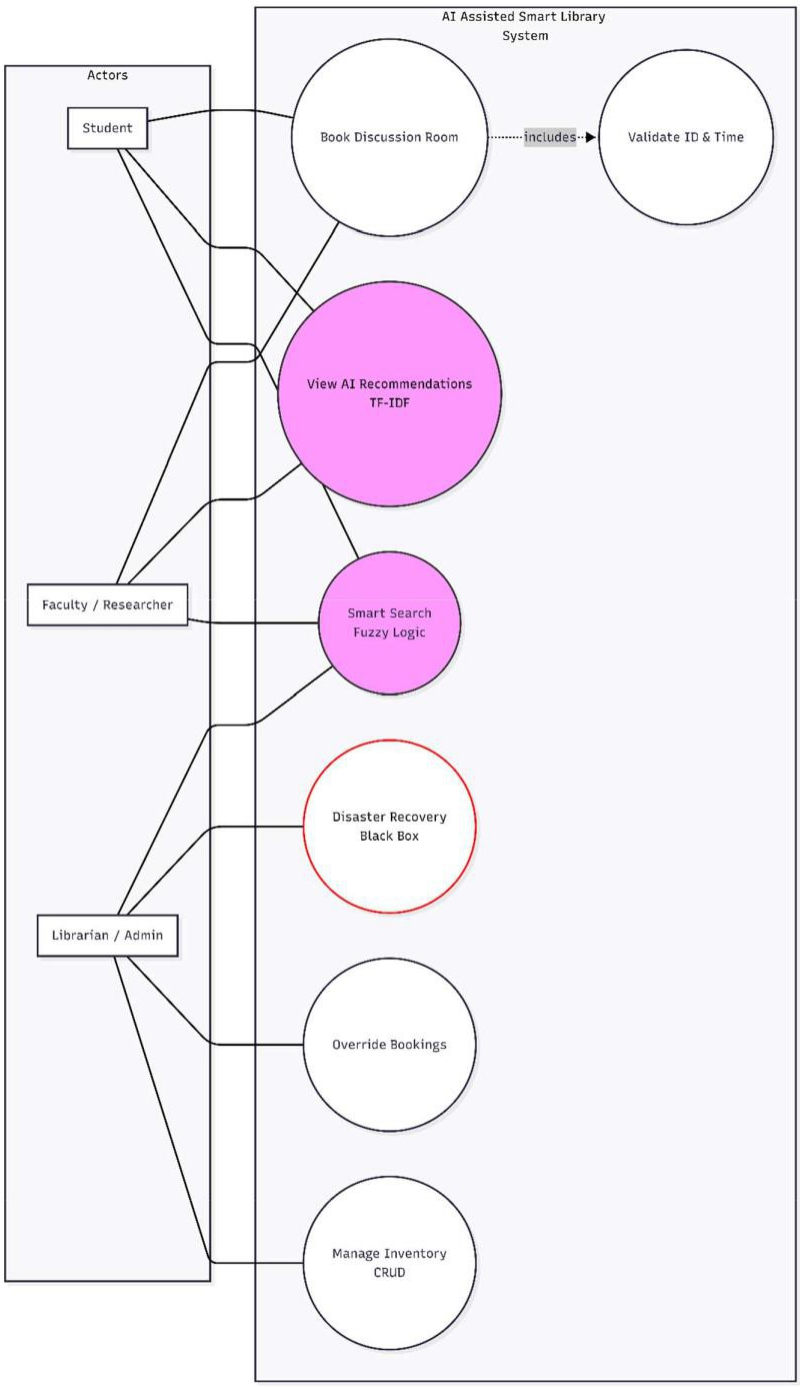

**Description:**

This diagram tracks the different states of a room booking slot during its lifecycle.

**Purpose:**

It manages the transition from `Free` to `Busy` and ensures that slots move to an `Expired` state once the **one- hour time limit** is reached.

## 5.USE_CASE Diagram:

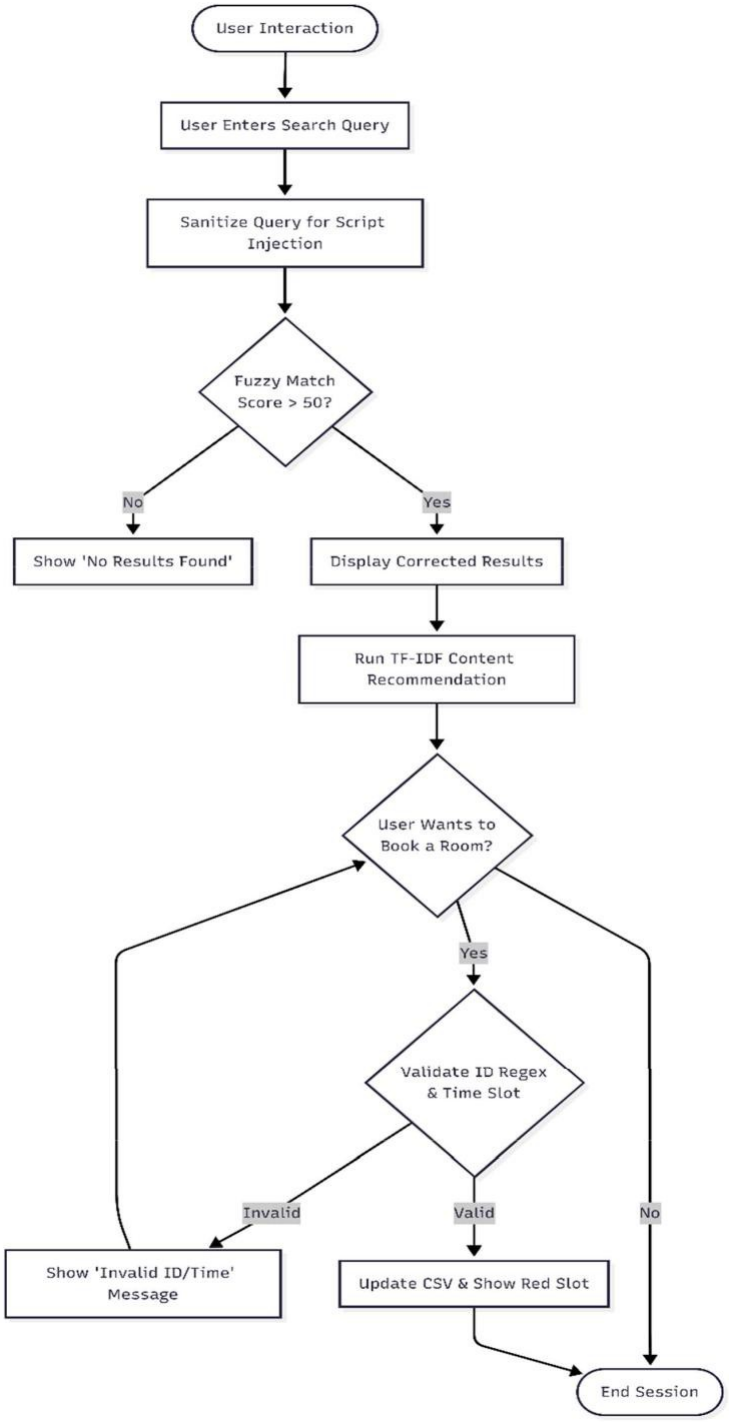



**Description:**

This diagram defines the functional scope of the system by illustrating how different actors—Students, Faculty, and Administrators—interact with specific features.

**Purpose:** It highlights the unique AI-driven services, such as **Fuzzy Logic search** and **TF-IDF recommendations**, alongside critical administrative functions like **Disaster Recovery**

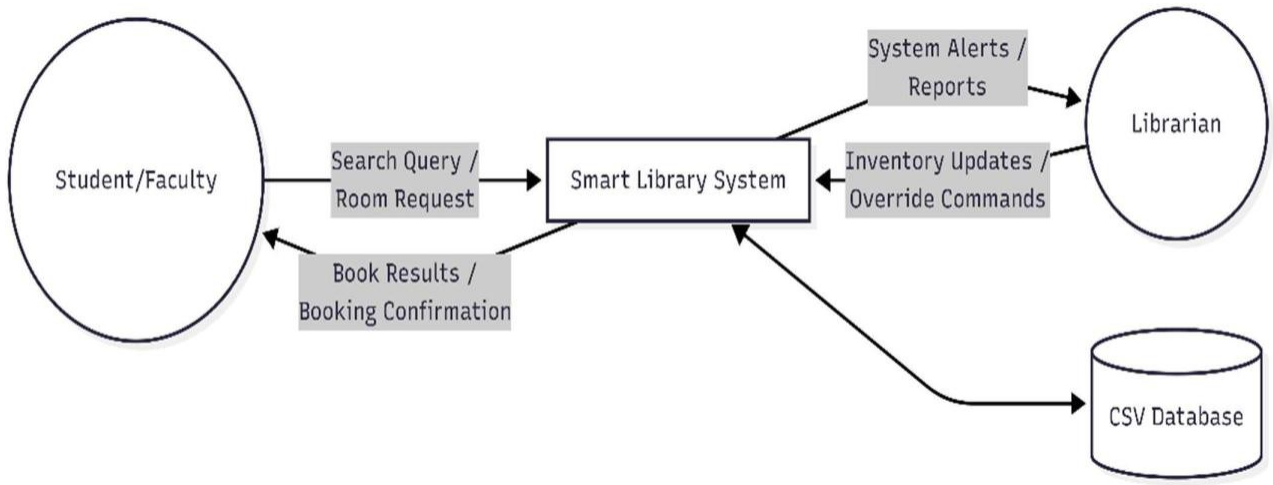
## 6. Activity Diagram for AI search:




**<u>Description:</u>** This dynamic model tracks the step-by-step logic of the system during a search and booking session.

**<u>Purpose:</u>** It specifically models the **Fuzzy Match threshold logic (Score > 50)** and the **Regex validation**
path required to finalize a room reservation

## 7.Data Flow Diagram:



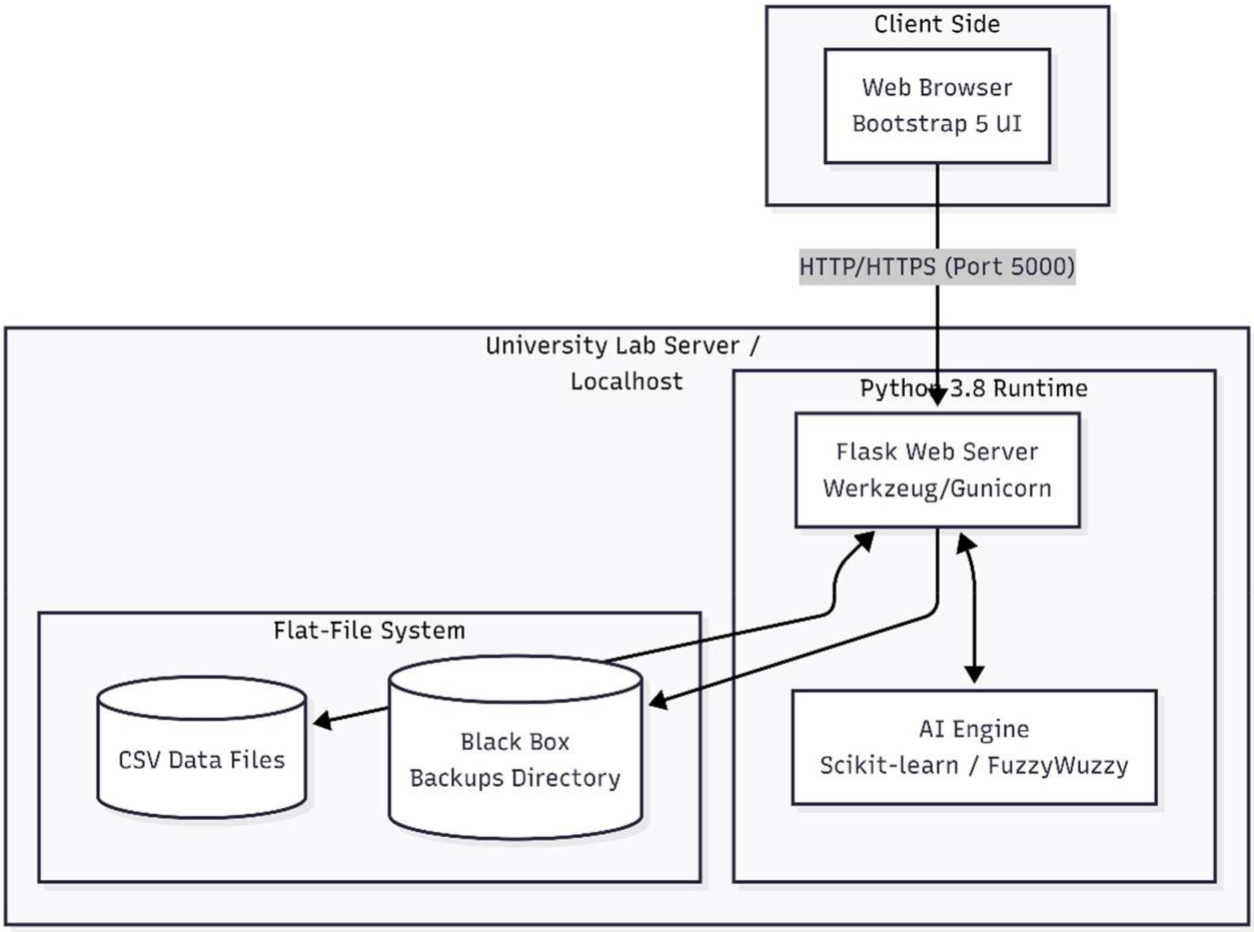

**Description:**

This diagram provides a high-level view of the entire data ecosystem.

**Purpose:**

It identifies the data flow between external entities (Students and Librarians) and the **Smart Library System**, showing exactly what information is input and output

## 8.Deployment Diagram:



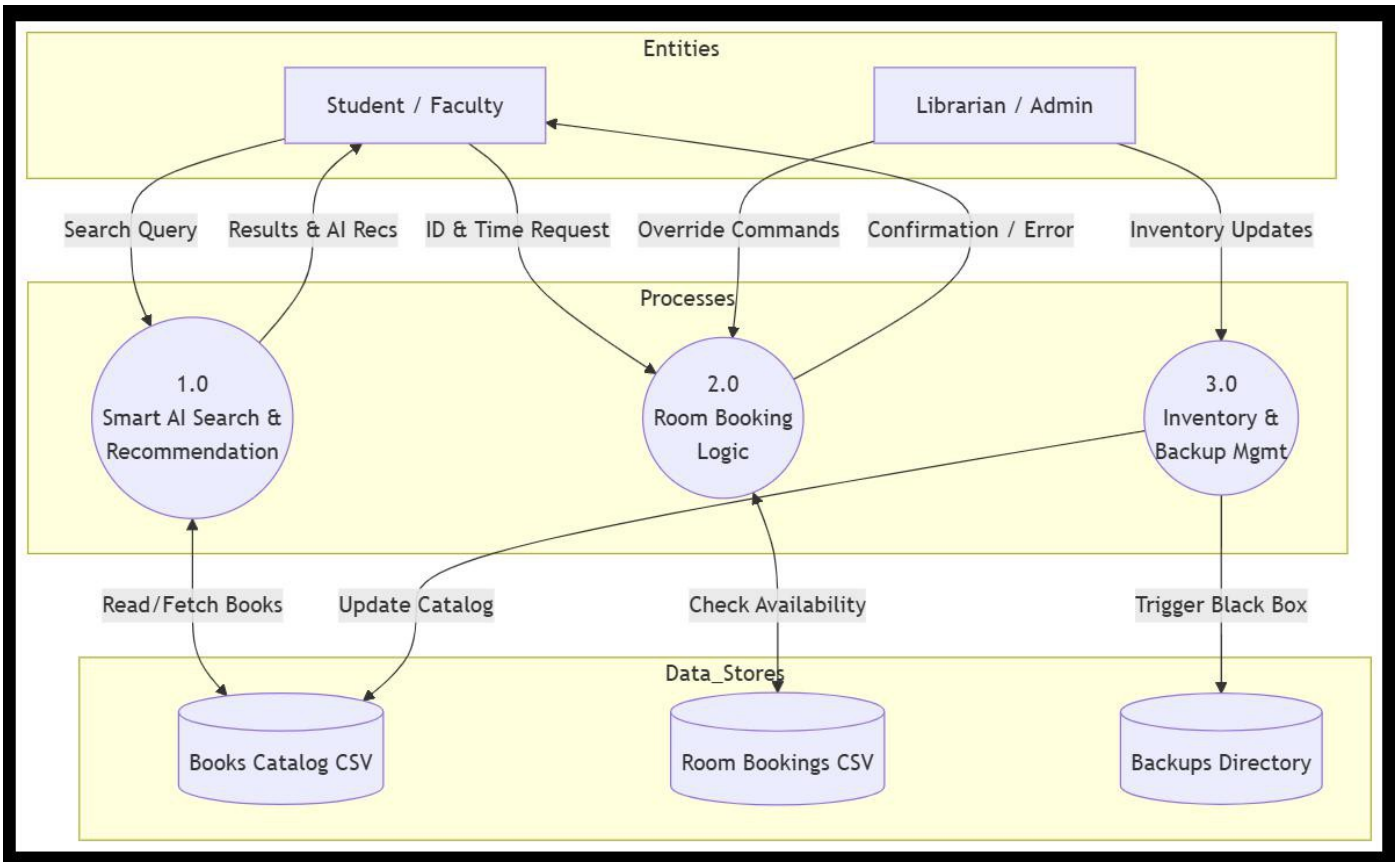

**Description:**

This model shows the physical hardware and software environment where the application is hosted.

**Purpose:**

It visualizes the connection between the **Bootstrap 5 client** and the **Python/Flask server**, detailing how they interact with the **Flat-File (CSV) storage system**

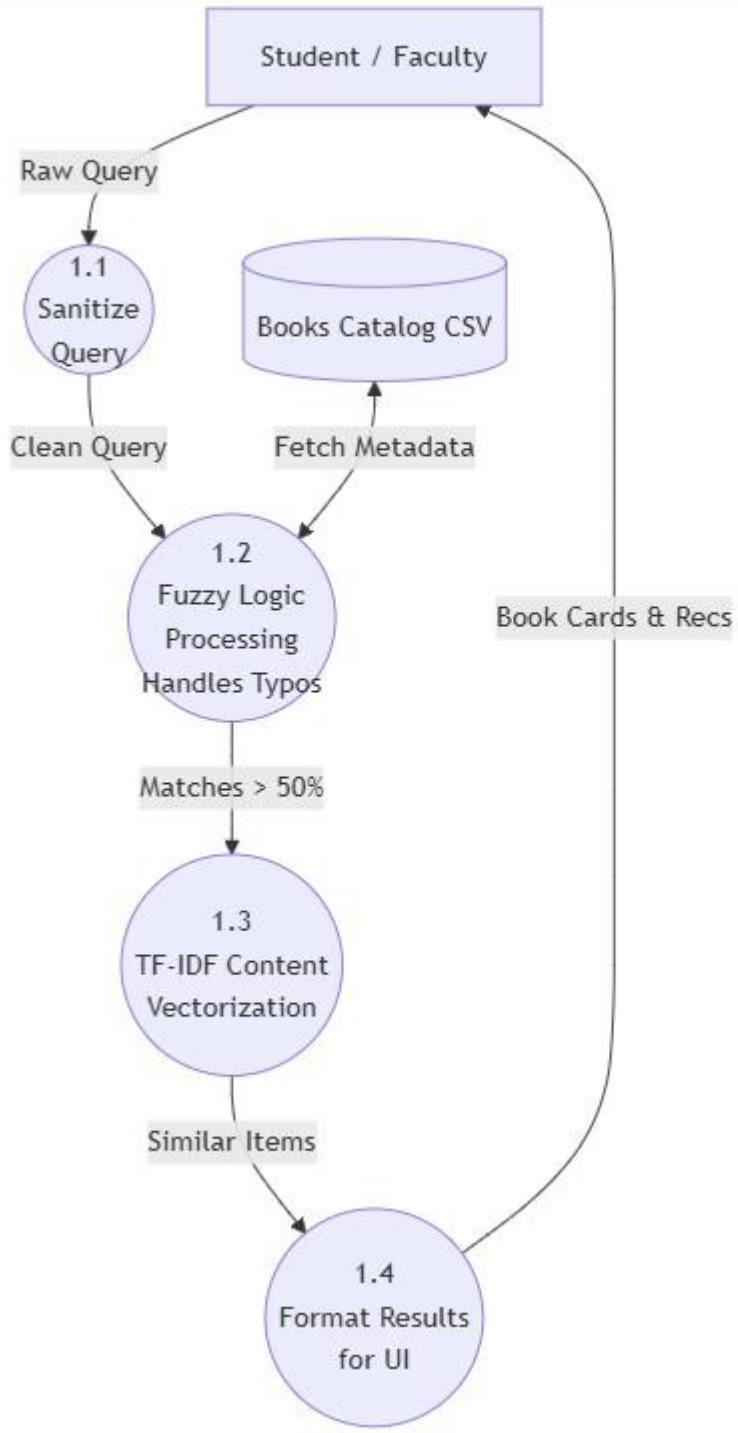
## 9.Data flow Diagram -level 1




**DFD Level 1 (Major Subsystems):**

Decomposes the central system into three distinct modules: AI-Driven Search, Resource Scheduling, and Administrative Management. It illustrates how the Student and Librarian actors interact with specific data stores like the Books Catalog and Room Bookings CSV files.

## 10.Data flow Diagram -level 2




**DFD Level 2 (AI Process Decomposition):**

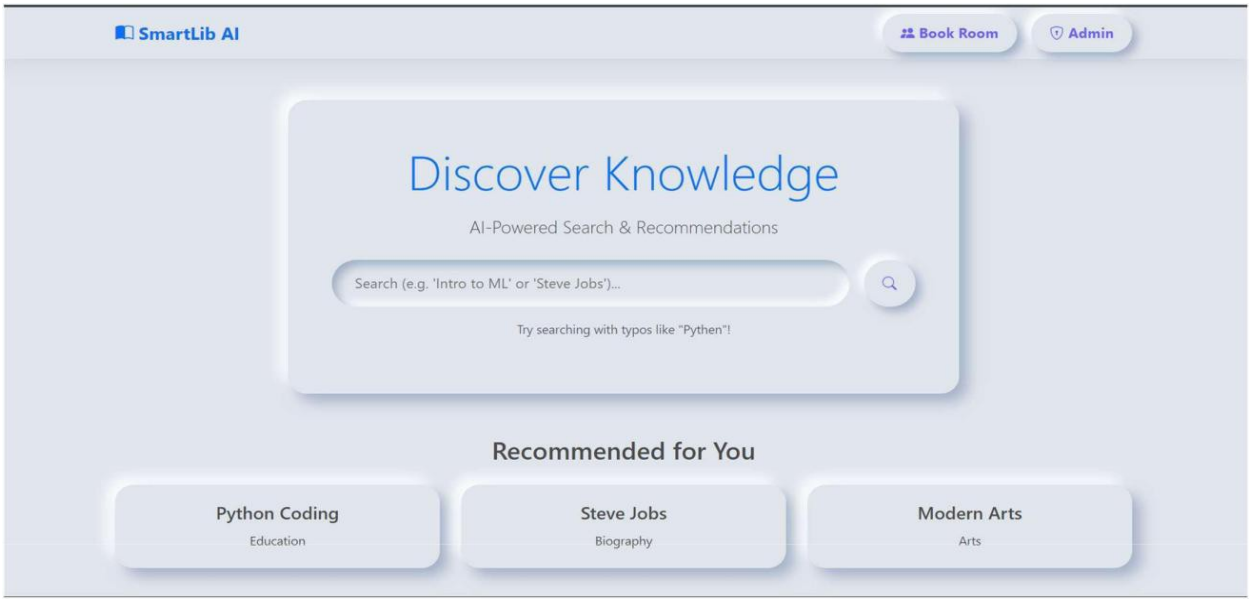
Provides a granular view of the **Search Engine (Process 1.0)**. It highlights the internal data transformations—from initial query sanitization to **Fuzzy Logic** matching and **TF-IDF** recommendation generation—that ensure high resource discoverability.

# 4. External Interface Requirements

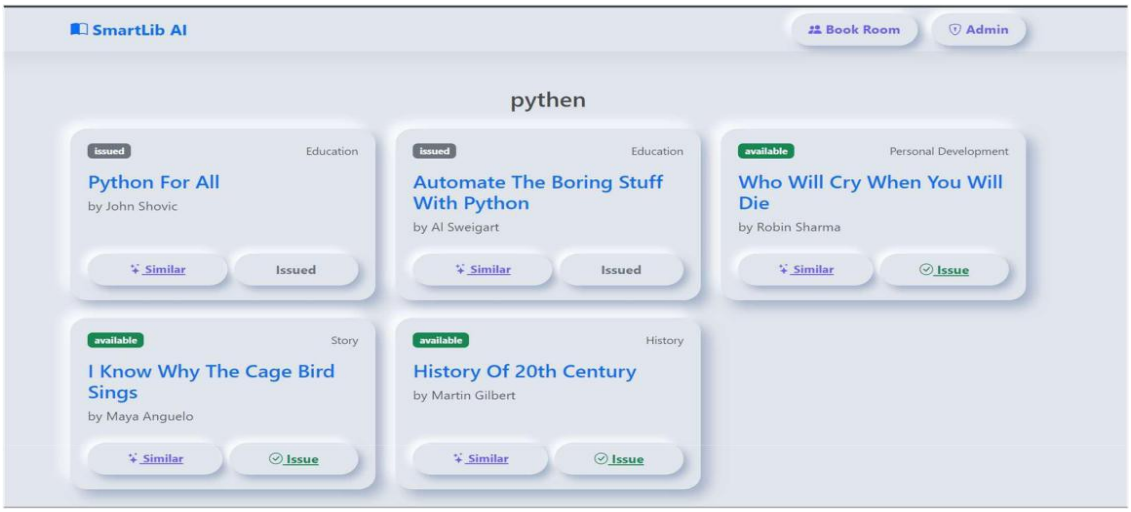
## 4.1 User Interfaces

The system features a Neumorphic (Soft UI) design language implemented via Bootstrap 5 and custom CSS.

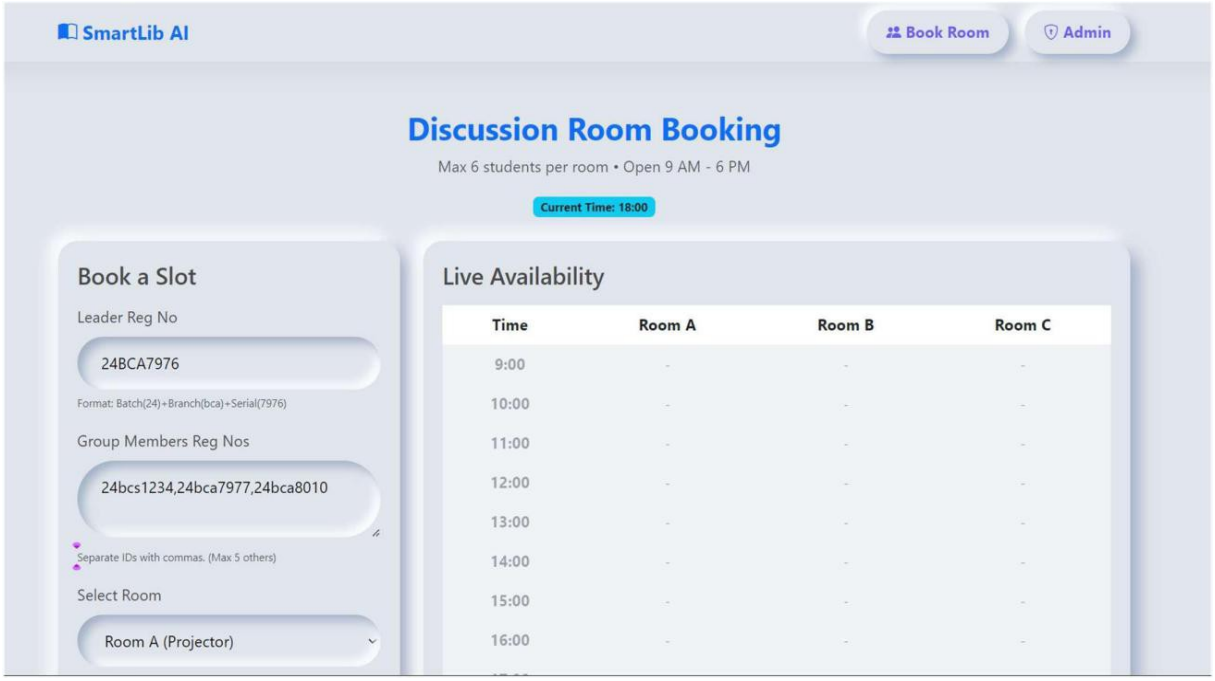
- Home Screen: Features a prominent "Smart Search" bar and "Quick Actions" for room booking.



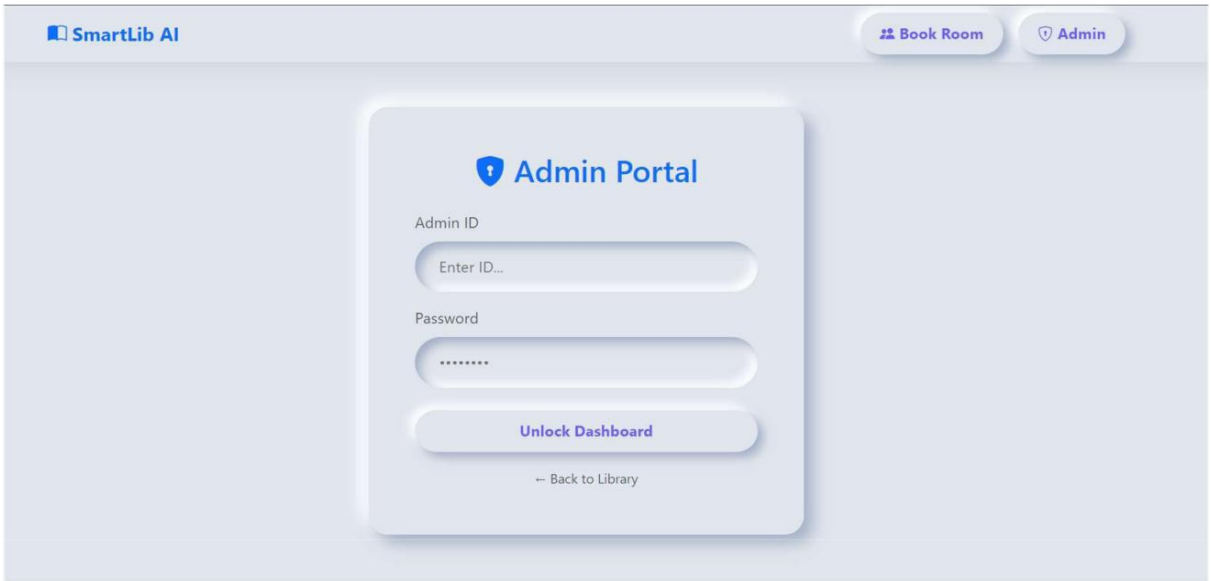

- Search Results Page: Displays books in a card layout. Each card contains "Title", "Author", "Status Badge", and a "Recommend Similar" button.



- Room Booking Grid: A responsive table showing time slots (Rows) vs. Rooms (Columns). "Busy" slots are visually distinct (Red) from "Free" slots (Green).




- Admin Dashboard: A dense data table view for inventory management with inline "Quick Action" buttons for status updates.



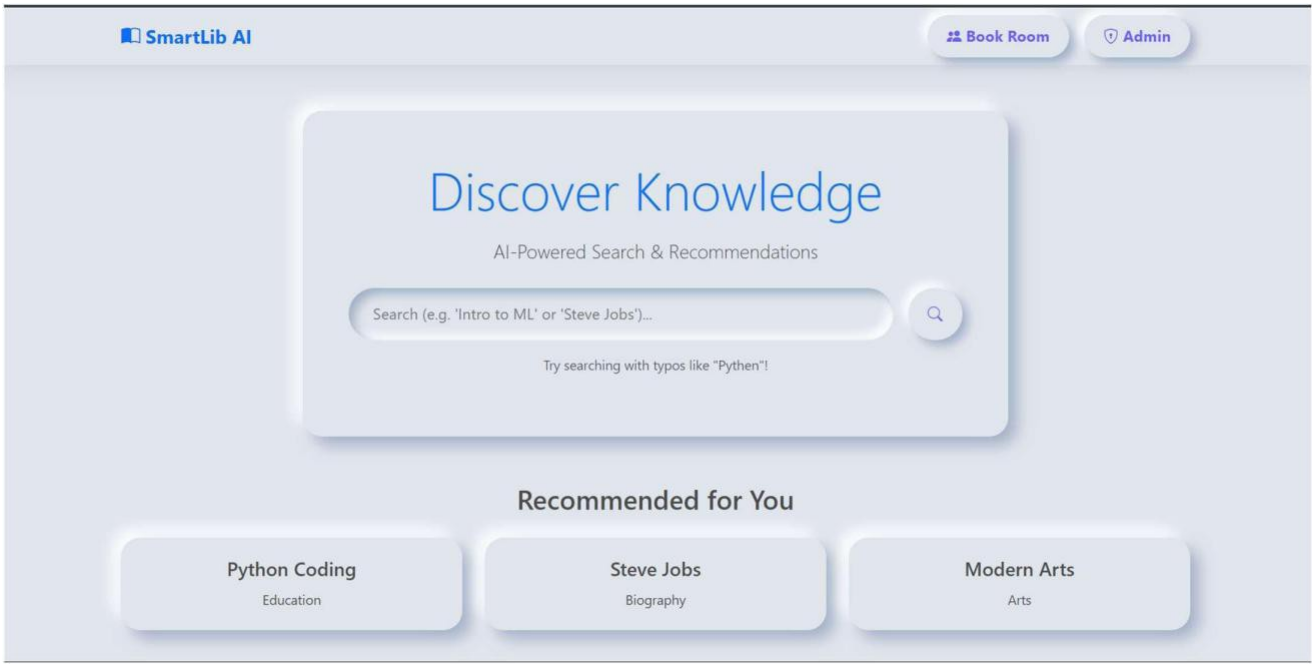
## 4.2 Hardware Interfaces

- Server Side: No specific hardware dependency; runs on standard x86/x64 architecture.
- Client Side: Responsive design supports Desktops (1920x1080), Laptops (1366x768), and Tablets

DESKTOP

## 4.3 Software Interfaces

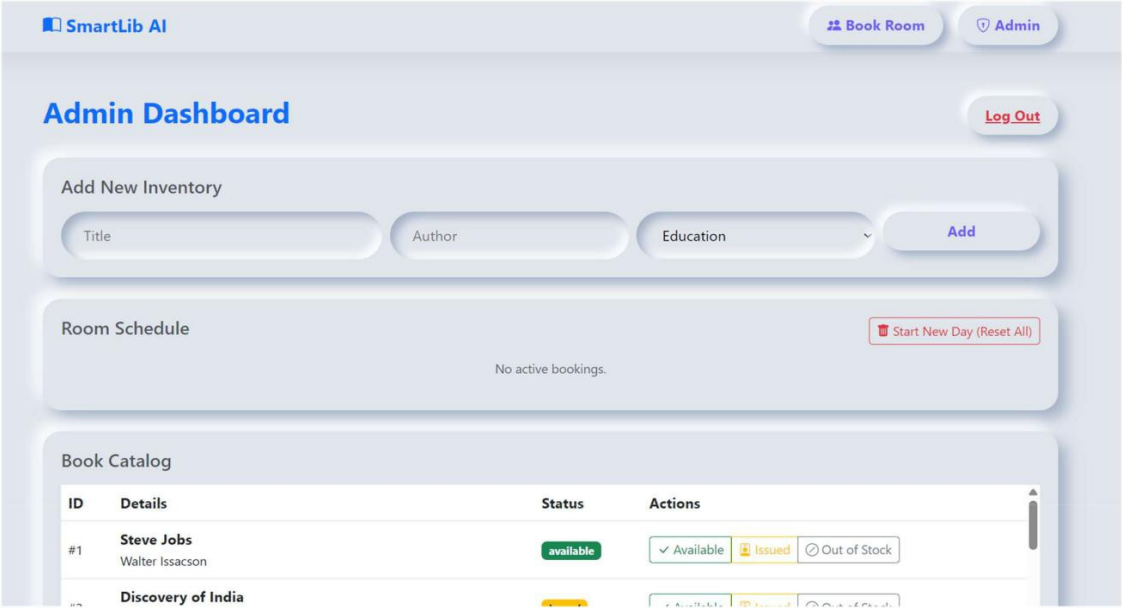
- Operating System: Cross-platform compatibility (Windows/Linux/macOS).
- Web Server: Werkzeug for Development ,and Gunicorn for Production.



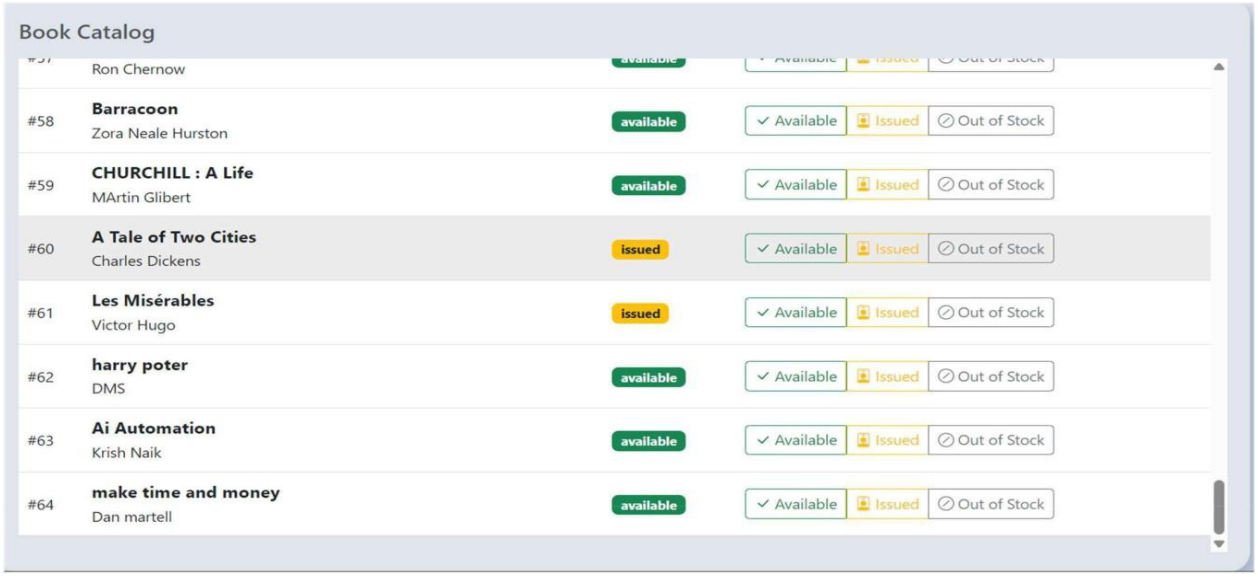

- Data Processing: Pandas DataFrames acting as an in-memory database interface.

## 4.4 Communication Interfaces

- Protocol: HTTP/HTTPS (Port 5000 default).
- Data Format: JSON is used for asynchronous (AJAX) communication between the Admin Frontend and the Flask Backend to update book statuses without page reloads.

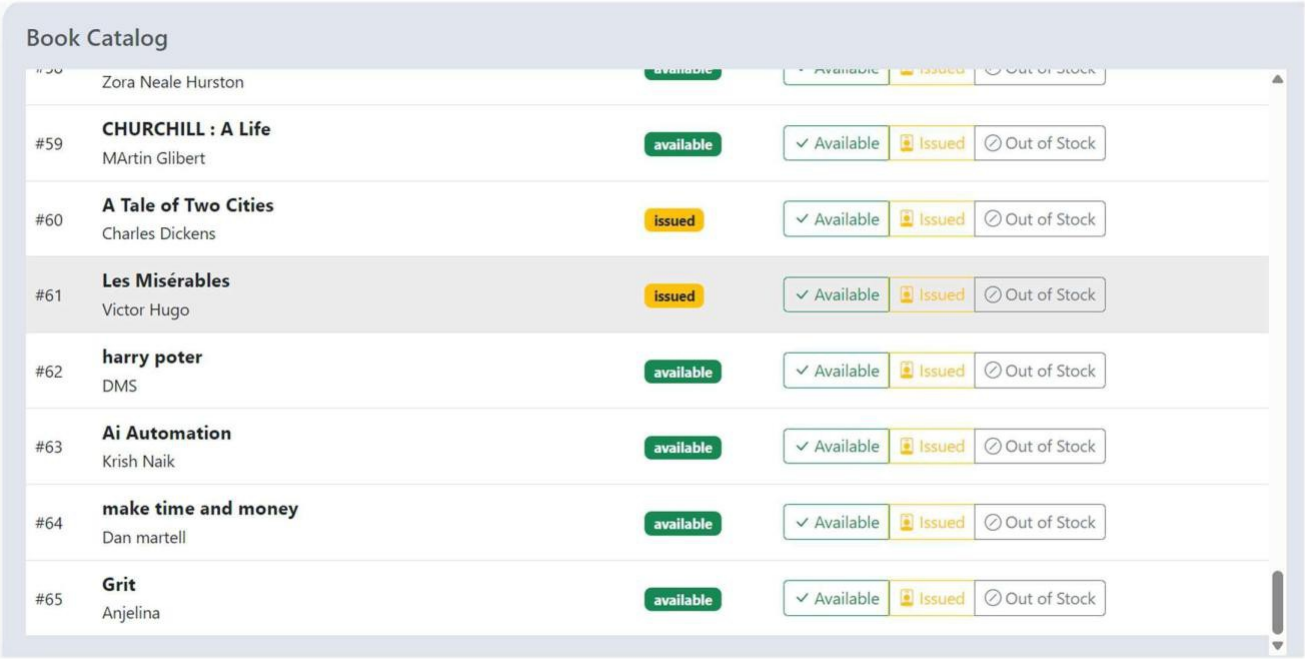



**Book Catalog:**



## Admin Dashboard

## Admin added new Book: Grit successfully



# 5.Other Non-Functional Requirements

## 5.1 Performance Requirements

- Search Latency: The AI search query must return results in under 2.0 seconds for a dataset of 5,000 books.
- Startup Time: The TF-IDF Vectorization model must train and initialize within 15 seconds of server boot.

## 5.2 Safety Requirements

- Data Preservation: The system must automatically create a timestamped copy of the bookings.csv file in the backups directory before any "Delete All" operation is executed.
- Fail-Safe: If the CSV file is corrupted or missing, the system must regenerate headers automatically to prevent crash loops.

## 5.3 Security Requirements

- Authentication: Administrative access is restricted via a session-based login requiring a **username (24bca7976) and password.**
- Input Validation:

  Student IDs are validated against the

  **Regex pattern: ^\d{2}[a-zA-Z]{3}\d{4}$.**

  Search queries are sanitized to prevent potential script

  injection.

- Access Control: Direct URL access to /admin routes without an active session is blocked and redirected to the login page.

### 5.4 Software Quality Attributes

- Robustness: The system handles edge cases
- e.g., booking a time slot that has already passed

with user-friendly error messages rather than stack traces.

- Usability: The "Fuzzy Search" feature ensures that users with low spelling proficiency can still utilize the library effectively.

### 5.5 Business Policies & Rules

- Concurrency Rule: A student cannot be part of two room bookings at the same time.
- Capacity Rule: A single discussion room cannot accommodate more than 6 students.
- Expiry Rule: Booking slots are valid for 1 hour; expired slots must visually "vanish" or be disabled in the UI.

# 6.Testing

## 6.1 Black-Box Testing

Focuses on input/output without looking at code.

| Test ID | Test Case Description | Input Data | Expected Result |
|---|---|---|---|
| BB-01 | Typo Tolerance Search | Query: "Pythen" | System displays "Python" related books. |
| BB-02 | Past Time Booking | Time: 10:00 AM (Current Time: 2:00 PM) | Error: "Cannot book past time slots." |
| BB-03 | Invalid ID Format | ID: "User123" | Error: "Invalid ID Format. Must match 24bca7976." |
| BB-04 | Double Booking | Student A books Room 1, then tries Room 2 | Error: "Student is already part of another booking." |
| BB-05 | Admin Login | User: 24bca7976, Pass: Wrong | Access Denied. Flash message displayed. |

### 6.2 White-Box Testing

Focuses on internal logic and code structure.

- WB-01 (Regex Logic): Verified the re.match() function correctly rejects strings with special characters or incorrect lengths in validate_booking().
- WB-02 (Vectorization): Verified that TfidfVectorizer correctly ignores English stop-words (e.g., "the", "and") during the matrix generation in ai_engine.py.
- WB-03 (File I/O): Verified that the create_backup() function correctly checks for the existence of the backups/ directory and creates it if missing before copying files.
- WB-04 (Levenshtein Distance): Tested the process.extract() threshold; confirmed that matches with a score < 50 are discarded to maintain relevant.

## 7. Conclusion

The "AI Assisted Smart Library System" successfully bridges the gap between traditional static library catalogues and modern, user-centric expectations. By integrating AI for "forgiving" search and contextual recommendations, the system significantly improves resource discoverability. Furthermore, the logic-driven booking system eliminates administrative overhead by enforcing rules automatically. The prototype demonstrates that intelligent features can be delivered in a lightweight, portable architecture, setting the stage for future scalability.

## 8.References


- Flask Documentation (v2.0): https://flask.palletsprojects.com/
- Scikit-Learn User Guide (TF-IDF): https://scikit-learn.org/
- FuzzyWuzzy Library: https://github.com/seatgeek/fuzzywuzzy
- Bootstrap 5 Documentation: https://getbootstrap.com/


## 9.Acknowledgements


I would like to express my sincere gratitude to my project guide, Pechetti Mounika, for their guidance and support throughout the development of this project. Their insights into software engineering principles were instrumental in shaping the architecture of this system. I also thank the School of Computer Science and Engineering for providing the necessary laboratory resources.